Why is Optimizing Mixture Distributions in PyTorch Still Such a Pain?

Alexander Udeneev, Alexey Ponomarev, Lina Gorbunova, Maxim Ivanov
MIPT

October 2026

1. The Deep Learning Dilemma with Multi-Modal Data

Picture this: you are training a state-of-the-art variational autoencoder or normalizing flow, and your data refuses to fit a nice, cozy unimodal Gaussian. Real-world uncertainty is messy, multimodal, and full of surprises. Naturally, you reach for **mixture distributions**.

Sampling from a mixture is straightforward—first draw a categorical component index, then sample from that specific component. But the moment you try to compute gradients via `loss.backward()`, PyTorch throws its hands up. Standard pathwise reparameterization fails because of the discrete component switch, and score-function estimators suffer from notoriously wild variance.

> **The Developer Pain:** Every time researchers need mixture gradients for custom objectives or ELBO optimization, they end up rewriting custom reparameterization tricks from scratch. We built **ReMix** to solve this once and for all with a clean, modular Python library.

2. The ReMix API Architecture

ReMix bridges the gap between complex probabilistic formulations and standard PyTorch modules. You define your mixture components, pass them into a unified estimator interface, and let the library handle the underlying math seamlessly.

Listing 1: Setting up a mixture-fitting objective with the current PoC API.

```
import torch
from torch.distributions import Normal

from remix import ELBOObjective, Mixture, ScoreFunction

components = [Normal(-2.0, 1.0), Normal(2.0, 1.0)]
target = Mixture(torch.tensor([0.2, 0.8]).log(), components)
logits = torch.nn.Parameter(torch.zeros(2))

objective = ELBOObjective(target.log_prob)
estimator = ScoreFunction(n_samples=512, baseline=0.0)
optimizer = torch.optim.Adam([logits], lr=0.05)
```

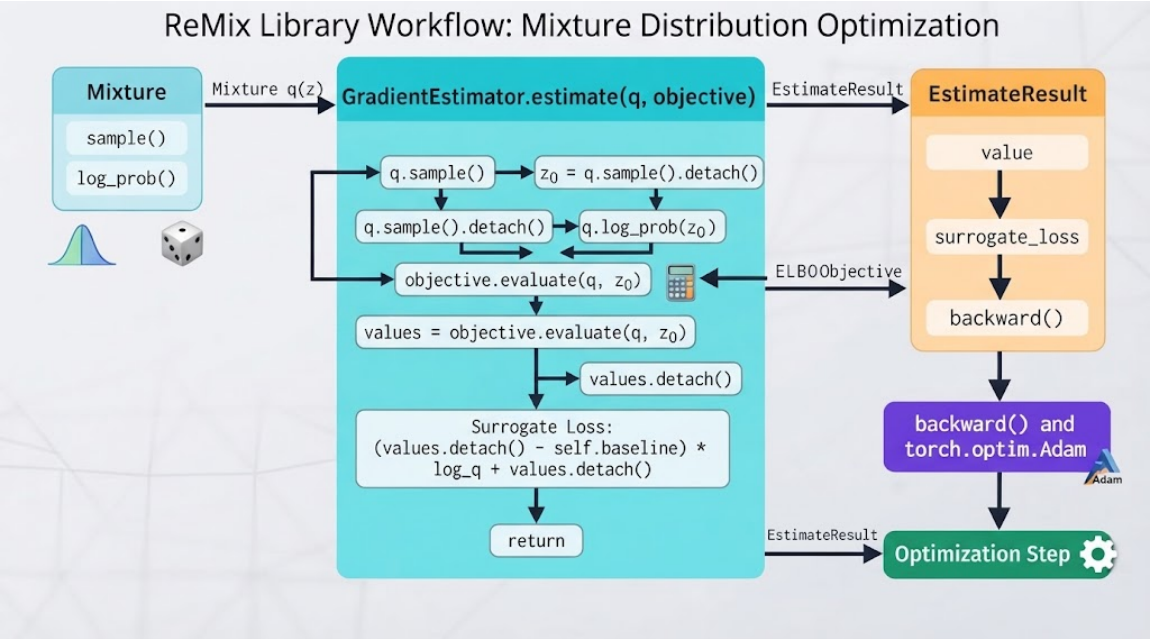


Figure 1: Under the hood of the ReMix API. The `Mixture` handles sampling and log-density evaluation. The `GradientEstimator` detaches the sample z_0, evaluates the objective, and constructs the `surrogate_loss` using the Score-Function STL formula. The `EstimateResult` keeps the detached ELBO value for logging and the surrogate loss for standard PyTorch backpropagation.

3. Inside the Engine: Score-Function with Sticking the Landing

To make gradient updates stable, ReMix implements advanced variance reduction techniques. The Score-Function estimator with the **Sticking the Landing (STL)** modification is expressed as:

$$g_{\text{STL}} = (f_\theta(Z) - b) \times \nabla_\theta \log q_\theta(Z)$$

Idea: By detaching the objective evaluation $f_\theta(Z)$ in the computational graph while retaining the score term, we eliminate unnecessary variance spikes while keeping unbiased updates.

Other Estimators: While our core prototype focuses on the Score-Function with STL, ReMix also supports Stratified estimators, Implicit Reparameterization (IR), and Post-Stratified estimators. You can explore the implementation details, mathematical derivations, and benchmarks for all alternative methods directly in our repository: `https://github.com/intsystems/re-mix`.

4. Benchmarks and Empirical Results

We evaluated ReMix on optimization tasks starting from flat priors $(0.5, 0.5)$ toward target weights $(0.2, 0.8)$ across 300 iterations:

- **Gaussian Mixture:** KL divergence decreased from $0.2001 \to 0.0000$. Learned weights: $[0.193, 0.807]$.
- **Laplace Mixture:** KL divergence decreased from $0.1624 \to 0.0003$. Learned weights: $[0.188, 0.812]$.

- **Mixed Family:** KL divergence decreased from $0.1755 \rightarrow 0.0001$. Learned weights: $[0.192, 0.808]$.

5. Get Started with ReMix Today

Whether you are tackling uncertainty estimation or complex generative modeling, ReMix brings order to mixture distributions. Check out our open-source repository and join the discussion on making probabilistic deep learning smoother!

References

1. J. Burroni and D. Sheldon. Gradient estimation for mixture variational inference. *Journal of Machine Learning Research*, 27(191):1-34, 2026.
2. G. Roeder, Y. Wu, and D. K. Duvenaud. Sticking the landing: Simple, lower-variance gradient estimators for variational inference. *NeurIPS*, 2017.
3. S. Mohamed et al. Monte Carlo gradient estimation in machine learning. *JMLR*, 21(132):1-62, 2020.